Lab 12

1. We've learned that SQL is generally a declarative language, but necessary additions have enabled it to be a procedural language in limited contexts. Based on what you know so far and without using procedural SQL, would it be possible to create a column where we can conditionally set an output value based on one or more input values? Use the space below to write out your answers or any ideas that you have.

2. The mechanism for assigning these conditional output values can happen through declarative SQL using `CASE WHEN` statements. In these statements we provide a certain set of conditions or a `CASE WHEN` and `THEN` assign an output value. The `CASE` is only used once at the beginning, while multiple `WHEN` and `THEN` statements can be used. A `CASE WHEN` statement always terminates with a catch-all `ELSE` and an `END`. Run the syntax below in a HeidiSQL window (as the root user). What do your results look like?

```
SELECT donation_status
       ,donation_status_desc
       ,CASE WHEN donation_status < 2 THEN 'status value < 2'
             ELSE 'status value > 2' END
FROM hfh.donation_status
```

3. Alter the `CASE WHEN` statement to make three mutually exclusive categories, one for less than or equal to two, one for greater than two and less than or equal to four and one for greater than four. It doesn't so much matter the values assigned, rather observing the results are most important. Ensuring the mutual exclusivity of `CASE WHEN` statements is extremely important and a common error. Make sure to explicitly assign a column name by including `AS column_name` after the `END`

4. We learned about tables, views and temporary tables, but sometimes we don't want these options, but we want a results set to carry through to use in a later part of our query. To do this we use something called a common table expression or CTE. CTEs enable us to create a results set that can be used multiple times within a query and then disappear when the query completes. Sometimes CTEs can be used simply to make our SQL code more legible, other times they can be used to improve query performance (by reusing a results set). Review the MariaDB documentation for CTEs, be sure to look at the examples on the linked Recursive and Non-recursive pages.

5. As the documentation describes, the `WITH <CTE NAME> AS(QUERY GOES HERE)` keyword is used to define a CTE. Alter the view that you created at the end of the last lab to perform only join two tables together in each CTE, then join the CTEs together to create the final results set.

6. Let's observe how we can use a single CTE twice. Select the `first_name`, `last_name` and `supporter_id` columns into a CTE named `supporters` from the `hfh.supporter` table. Use these results to perform separate joins to the `hfh.email_address` and `hfh.donation` tables into CTEs. Join those CTEs together into a single results set. To create addition CTEs, start the new queries with `,<CTE NAME> AS(QUERY GOES HERE)`

7. Putting it altogether now! Using the `hfh.podcast_subscription` table we want to breakdown subscriber counts into three categories: early adopters (less than 6 months), majority (between 6 months and one year) and late adopters (later than one year). All of the dates should be calculated relative to the start date of the podcast. Any podcasts that are less than one year old should be excluded.